

Paper–Code Comparison Report

Reach-Avoid Differential Game with Reachability Analysis for UAVs
Bui et al. (arXiv:2512.22793) vs. Codebase Implementation

Auto-generated Analysis

March 26, 2026

Contents

1	Introduction	3
2	Defender Reachability Algorithms	3
2.1	Vertical Invariant Capture Set ($\mathcal{B}_z$)	3
2.1.1	Paper (Section IV.A)	3
2.1.2	Code Implementation	3
2.1.3	Discrepancies	4
2.2	Horizontal Invariant Capture Set ($\mathcal{B}_h$)	4
2.2.1	Paper (Section IV.B)	4
2.2.2	Code Implementation	4
2.2.3	Discrepancies	4
2.3	Vertical Reach-Avoid Game (Φ_z)	5
2.3.1	Paper (Section V.A)	5
2.3.2	Code Implementation	5
2.3.3	Discrepancies	5
2.4	Horizontal Reach-Avoid Game (Φ_h)	5
2.4.1	Paper (Section V.B)	5
2.4.2	Code Implementation	5
2.4.3	Discrepancies	6
3	Defender Optimal Control	6
3.1	Algorithm 1: Vertical Reach-Track	6
3.1.1	Paper (Algorithm 1, p. 17)	6
3.1.2	Code Implementation	6
3.1.3	Discrepancies	7
3.2	Algorithm 2: Horizontal Reach-Track-Avoid	7
3.2.1	Paper (Algorithm 2, p. 18)	7
3.2.2	Code Implementation	7
3.2.3	Discrepancies	7
3.3	Control Extraction from Gradients	7
3.3.1	Paper	7
3.3.2	Code Implementation	8
3.3.3	Discrepancies	8

4 Attacker Optimal Control	8
4.1 Attacker Reaching (T_{goal} Computation)	8
4.1.1 Paper (Section VI, Eq. 40–42)	8
4.1.2 Code Implementation	8
4.1.3 Discrepancies	9
4.2 Attacker Escape Control (Online)	9
4.2.1 Paper	9
4.2.2 Code Implementation	9
4.2.3 Discrepancies	9
5 Winning Condition Analysis	9
5.0.1 Paper (Section VI)	9
5.0.2 Code Implementation	10
5.0.3 Discrepancies	10
6 Wall Avoidance Safety Layer	10
7 Summary of Discrepancies	11
8 Recommended Fixes (Prioritized)	11
8.1 Priority 1: Fix Horizontal Game Formulation	11
8.2 Priority 2: Fix Control Constraint Geometry	11
8.3 Priority 3: Use 6D Horizontal Max Distance	11
8.4 Priority 4: Improve $T_{\text{goal}}/T_{\text{capture}}$ Accuracy	11
8.5 Priority 5: Address Wall Avoidance Integration	12
9 Conclusion	12

1 Introduction

This document provides a systematic comparison between the algorithms described in:

Bui, M., Monckton, S., and Chen, M. “Reach-Avoid Differential Game with Reachability Analysis for UAVs: A Decomposition Approach,” *arXiv:2512.22793v1*, December 2025.

and their implementation in the project codebase. For each algorithm, we present:

1. The paper’s mathematical formulation with equation references
2. The corresponding code implementation with file/line references
3. Identified discrepancies and their severity
4. Recommended fixes

Severity levels:

- **CRITICAL** – Likely cause of incorrect game outcomes
- **MODERATE** – May degrade accuracy or break guarantees
- **MINOR** – Numerical or practical differences

2 Defender Reachability Algorithms

2.1 Vertical Invariant Capture Set ($\mathcal{B}_z$)

2.1.1 Paper (Section IV.A)

The paper defines relative vertical dynamics (Eq. 29):

$$\dot{\mathbf{x}}_z^{\text{rel}} = \begin{bmatrix} \dot{z}^{\text{rel}} \\ \dot{v}_D^z \end{bmatrix} = \begin{bmatrix} v_D^z - v_A^{z,C} \\ k_z(v_D^{z,C} - v_D^z) \end{bmatrix} \quad (1)$$

The maximum distance value function (Eq. 30):

$$V_z(\mathbf{x}_z^{\text{rel}}, t) = \sup_{\mathbf{u}_A^z(\cdot)} \inf_{\mathbf{u}_D^z(\cdot)} \max_{\tau \in [0, t]} |z^{\text{rel}}(\tau)| \quad (2)$$

The invariant set: $\mathcal{B}_z = \{\mathbf{x}_z^{\text{rel}} \mid V_{z,\infty}(\mathbf{x}_z^{\text{rel}}) \leq d_z\}$.

2.1.2 Code Implementation

`vertical_relative.py`: Implements Eq. (1) correctly.

```
# State: [z_rel, v_D_z]
open_loop: [state[1], -k_z * state[1]] # [v_Dz, -k_z*v_Dz]
control_jacobian: [[0.0], [k_z]] # u_D enters v_Dz_dot
disturbance_jacobian: [[-1.0], [0.0]] # d_A enters z_rel_dot
```

`vertical_solver.py`:152-209: Solves $V_{z,\infty}$ with:

- Initial condition: $|z^{\text{rel}}|$ (correct per paper)
- Solver mode: `maxVWithV0` (implements $\max_{\tau}$ in Eq. (2))
- Time horizon: $T = 20$ s with 200 steps (sufficient for convergence)
- Grid: 2D $[z^{\text{rel}}, v_D^z]$ per paper

`vertical_solver.py`:211-251: Extracts $\mathcal{B}_z$:

```
v_min = float(vf.values.min())
d_z_effective = max(d_z, v_min * 1.05) # <-- WORKAROUND
b_z_mask = (vf.values <= d_z_effective).astype(np.float64)
```

2.1.3 Discrepancies

- **MODERATE: Effective threshold fallback.** When grid resolution is coarse, $\min(V_{z,\infty}) > d_z$, so the code relaxes the threshold to $d_z^{\text{eff}} = 1.05 \cdot \min(V_{z,\infty})$. This means $\mathcal{B}_z$ may be *larger* than mathematically warranted, weakening the invariance guarantee. **Fix:** Increase grid resolution so $\min(V_{z,\infty}) \leq d_z$, or warn when fallback is triggered.

Overall: Dynamics and solver formulation are **correct**. The fallback threshold is the only concern.

2.2 Horizontal Invariant Capture Set ($\mathcal{B}_h$)

2.2.1 Paper (Section IV.B)

The paper defines (Eq. 34):

$$V_h(\mathbf{x}_h, t) = \sup_{\mathbf{u}_A^h(\cdot)} \inf_{\mathbf{u}_D^h(\cdot)} \max_{\tau \in [0, t]} l_h\left(\zeta_{\mathbf{x}_h, 0}^{\mathbf{u}_D^h(\cdot), \mathbf{u}_A^h(\cdot)}(\tau)\right) \quad (3)$$

where $l_h(\mathbf{x}) = \max\{l_h^{\text{rel}}(\mathbf{x}), o_h(\mathbf{x})\}$ and $o_h(\mathbf{x}) = K$ if $(x_D, y_D) \in \Omega_{\text{obs}}$, else 0.

Key point: V_h is defined over the **full 6D horizontal state** $\mathbf{x}_h = (x_D, y_D, v_D^x, v_D^y, x_A, y_A)$ because the obstacle penalty o_h depends on *absolute* defender position, not relative coordinates.

2.2.2 Code Implementation

`horizontal_solver.py:279-339`: Default `solve_horizontal_max_distance` uses **4D relative dynamics**:

```
# State: [x_rel, y_rel, v_D_x, v_D_y] (4D, NO absolute positions)
grid = create_horizontal_relative_grid(config)
dynamics = HorizontalRelativeDynamics(config)
initial_values = sqrt(x_rel^2 + y_rel^2) # No obstacle penalty!
```

An alternative `solve_horizontal_max_distance_6d` (lines 473–535) does exist but is not the default pipeline.

2.2.3 Discrepancies

- **CRITICAL:** $V_{h,T}$ **computed in 4D without obstacles.** The default solver uses 4D relative dynamics, which *cannot* encode the obstacle penalty $o_h(\mathbf{x})$ from Eq. (3). The paper’s V_h penalizes the defender for entering obstacles during tracking; the code’s $V_{h,T}$ does not. Consequence: $\mathcal{B}_h$ may include states where the defender collides with obstacles while tracking.
Fix: Use the existing `solve_horizontal_max_distance_6d` as the default, or integrate obstacle penalties into the 4D formulation via a conservative approximation (e.g., penalty when relative position implies defender is near obstacles given attacker position).
- **MODERATE:** $V_{h,T}$ **does not converge** ($T = 2.5\text{s}$). Both paper and code acknowledge this. $\mathcal{B}_h$ provides only a 2.5-second tracking guarantee, *not* indefinite invariance. This is a fundamental limitation of the parameters, not a code bug.
- **MODERATE:** Same d_h^{eff} fallback as vertical (relaxes threshold when coarse grid).

2.3 Vertical Reach-Avoid Game (Φ_z)

2.3.1 Paper (Section V.A)

The paper modifies the reach set (Eq. 38):

$$\mathcal{R}_z = \{\mathbf{x}^z \mid \mathbf{x}_z^{\text{rel}} \in \mathcal{B}_z\} \quad (4)$$

and solves Eq. 25 with $l_z(\mathbf{x}^z) = V_{z,\infty}(z^{\text{rel}}) - d_z$ as the level-set representation of $\mathcal{B}_z$.

In this game, the **defender minimizes** Φ_z (Eq. 26a) while the **attacker maximizes** (Eq. 26b).

2.3.2 Code Implementation

`vertical_solver.py:79-150`: When $V_{z,\infty}$ is available, uses $\mathcal{B}_z$ as target (correct per Eq. 38).

`vertical_game.py`: Sets `control_mode="max"` (defender) and `disturbance_mode="min"` (attacker).

2.3.3 Discrepancies

- **MINOR: Optimization direction convention.** The paper states the defender *minimizes* Φ_z (Eq. 26a). The code sets `control_mode="max"`. These are *equivalent* because the `hj_reachability` library convention defines the sub-zero level set as the backward reachable tube, and the maximizer is the player trying to *reach* the target. Since the target is the capture set, the defender (maximizer) reaching capture $\equiv$ defender minimizing distance. The sign of Φ_z is simply flipped relative to the paper. **No fix needed** – conventions are internally consistent.

Overall: Vertical reach-avoid is **correctly implemented**.

2.4 Horizontal Reach-Avoid Game (Φ_h)

2.4.1 Paper (Section V.B)

The paper defines reach and avoid sets (Eq. 39):

$$\mathcal{R}_h = \{\mathbf{x}^h \mid \mathbf{x}_A^h \in \mathcal{T}\} \cup \{\mathbf{x}^h \mid \mathbf{p}_D^h \in \Omega_{\text{obs}}\} \quad (5)$$

$$\mathcal{A}_h = \{\mathbf{x}^h \mid \mathbf{x}_h^{\text{rel}} \in \mathcal{B}_h \wedge \mathbf{x}_A^h \notin \mathcal{T}\} \cup \{\mathbf{x} \mid \mathbf{x}_A \in \Omega_{\text{obs}}\} \quad (6)$$

Key: The reach set $\mathcal{R}_h$ includes *the attacker reaching the target* $\mathcal{T}$. The game value function Φ_h therefore accounts for the interaction between the defender trying to capture and the attacker trying to reach the goal, *within a single PDE solve*.

The winning regions (Eq. 22):

$$\mathcal{W}_{A,h} = \{\mathbf{x}^h \mid \Phi_h(\mathbf{x}^h) \leq 0\} \quad (\text{attacker wins}) \quad (7)$$

$$\mathcal{W}_{D,h} = \{\mathbf{x}^h \mid \Phi_h(\mathbf{x}^h) > 0\} \quad (\text{defender wins}) \quad (8)$$

2.4.2 Code Implementation

`horizontal_solver.py:202-277`:

```
# Target: capture set only (NO attacker target region T)
target_values = _make_horizontal_capture_set(grid, config.capture.d_h)

# Avoid: walls + obstacles + complement(B_h) + attacker obstacles
obstacle_values = _make_avoid_set_with_Bh(grid, config, v_h_t_data, d_h)
attacker_obstacles = _make_attacker_obstacle_avoid_set(grid, config)
obstacle_values = np.minimum(obstacle_values, attacker_obstacles)
```

winning_conditions.py:187:

```
in_w_d_h = phi_h_val <= 0 # Defender wins when phi_h <= 0
```

2.4.3 Discrepancies

- **CRITICAL: Target region $\mathcal{T}$ missing from reach set.** The paper’s $\mathcal{R}_h$ (Eq. 5) includes $\{\mathbf{x}_A^h \in \mathcal{T}\}$, meaning the value function Φ_h directly encodes whether the attacker can reach its goal while the defender tries to capture. The code only uses the *capture set* as the BRT target and handles goal-reaching *separately* via T_{goal} timing analysis.

Impact: The code’s winning region $\mathcal{W}_{D,h}$ does not account for the attacker’s goal-reaching ability within the game. States where the attacker can reach $\mathcal{T}$ before capture may be incorrectly classified as defender-winning.

Fix: Construct the reach set as per Eq. 5:

```
# Reach set (for attacker): target_region OR defender_in_obstacle
l_T = make_target_region_sdf(grid, config) # attacker in T
l_obs = make_defender_obstacle_sdf(grid, config) # defender in obs
reach_set = np.minimum(l_T, l_obs)
```

- **CRITICAL: Winning region sign convention.** The paper defines $\mathcal{W}_{D,h} = \{\Phi_h > 0\}$ (Eq. 22b). The code uses $\Phi_h \leq 0$ for defender winning. This inversion arises because the code solves a *different* problem (capture-as-target from defender’s perspective) rather than the paper’s formulation (attacker reaching goal). The two formulations would give different value functions.

Fix: If the reach set is corrected per the item above, the winning region convention must also match the paper: $\mathcal{W}_{D,h} = \{\Phi_h > 0\}$.

- **MODERATE: Avoid set missing $\mathbf{x}_A^h \notin \mathcal{T}$ condition.** Per Eq. 6, capture only counts as an avoid event (for the attacker) when the attacker is *outside* $\mathcal{T}$. If the attacker is inside $\mathcal{T}$ at capture time, the attacker still wins. The code’s avoid set does not enforce this condition.

Fix: Modify the avoid set construction:

```
# g_capture: negative inside capture set
# g_not_T: negative when attacker NOT in T (i.e., positive in T)
g_not_T = -make_target_region_sdf_attacker(grid, config)
avoid_capture = np.maximum(g_capture, g_not_T) # intersection
```

3 Defender Optimal Control

3.1 Algorithm 1: Vertical Reach-Track

3.1.1 Paper (Algorithm 1, p. 17)

1. While $\mathbf{x}^z \in \mathcal{W}_{D,z}$:
2. If $\mathbf{x}_z^{\text{rel}} \notin \mathcal{B}_z$: Apply optimal reaching control from Φ_z (Eq. 26)
3. Else if deep inside $\mathcal{B}_z$ ($V_{z,\infty} - d_z \leq \epsilon$): Use performant tracker (e.g., PID)
4. Else (near boundary of $\mathcal{B}_z$): Apply optimal tracking from $V_{z,\infty}$ (Eq. 33)

3.1.2 Code Implementation

defender_node.py:193-247: Implements all three modes with additions:

- Checks winning region before applying optimal control

- Falls back to PID pursuit when outside $\mathcal{W}_{D,z}$
- Uses margin-based threshold: `near_boundary = V_z_inf > (d_z_eff - margin)`

3.1.3 Discrepancies

- **MINOR: PID fallback outside winning region.** Paper’s Algorithm 1 only operates while $\mathbf{x}^z \in \mathcal{W}_{D,z}$. The code adds a PID pursuit mode when outside the winning region. This is pragmatically useful but *has no theoretical guarantee*.
- **MINOR: Deep-inside threshold differs.** Paper: $V_{z,\infty}(\mathbf{x}_z^{\text{rel}}) - d_z \leq \epsilon$ (small ϵ). Code: $V_{z,\infty} \leq d_z^{\text{eff}} - 0.3 \cdot d_z$. The code uses a larger margin (30% of d_z), meaning it switches to PID tracking *earlier*. This is conservative but safe.

Overall: Algorithm 1 is **correctly implemented** with pragmatic additions.

3.2 Algorithm 2: Horizontal Reach-Track-Avoid

3.2.1 Paper (Algorithm 2, p. 18)

1. **While** $\mathbf{x}^h \in \mathcal{W}_{D,h}$:
2. **If** $\mathbf{x}_h^{\text{rel}} \notin \mathcal{B}_h$: Apply optimal reaching from Φ_h (Eq. 21)
3. **Else if** deep inside $\mathcal{B}_h$: Use performant tracker
4. **Else**: Apply horizontal tracking from $V_{h,\infty}$ (Eq. 37)

3.2.2 Code Implementation

`defender_node.py:275-333`: Same structure as vertical with PID fallbacks.

3.2.3 Discrepancies

Same as Algorithm 1 (PID fallback, threshold differences). No additional issues beyond the Φ_h formulation problems described in Section 2.4.

3.3 Control Extraction from Gradients

3.3.1 Paper

Defender optimal controls (Eqs. 21a, 26a, 33, 37) all follow:

$$\mathbf{u}_D^* = \text{argopt}_{\mathbf{u}_D} \left(\frac{\partial \Phi}{\partial \mathbf{x}} \right)^\top f(\mathbf{x}, \mathbf{u}_A, \mathbf{u}_D) \quad (9)$$

where “opt” is max or min depending on the specific game.

For the double integrator, the control enters linearly through the velocity dynamics:

$$v_{D,\text{dot}}^x = k_x(u_x - v_D^x) \implies \text{Hamiltonian contribution} = \frac{\partial \Phi}{\partial v_D^x} \cdot k_x \cdot u_x \quad (10)$$

Bang-bang optimal control:

$$u_x^* = U_D^h \cdot \text{sign} \left(\frac{\partial \Phi}{\partial v_D^x} \cdot k_x \right) \quad (\text{for maximization}) \quad (11)$$

3.3.2 Code Implementation

horizontal_game.py:65-68:

```
def opt_ctrl_numpy(self, state, spat_deriv):
    u_x = np.where(spat_deriv[2] * self.k_x >= 0, self.u_d_h, -self.u_d_h)
    u_y = np.where(spat_deriv[3] * self.k_y >= 0, self.u_d_h, -self.u_d_h)
```

3.3.3 Discrepancies

- **CRITICAL:** Control input constraints are box, not circular. The paper specifies (Eq. 17a):

$$\sqrt{(v_D^{x,C})^2 + (v_D^{y,C})^2} \leq U_D^h \quad (12)$$

The code uses a **box constraint**: $|v_D^{x,C}| \leq U_D^h$ and $|v_D^{y,C}| \leq U_D^h$ independently. This allows a maximum diagonal speed of $\sqrt{2} \cdot U_D^h \approx 8.49$ m/s instead of the paper's 6 m/s. The same applies to the attacker ($\sqrt{2} \cdot 3 \approx 4.24$ m/s vs. 3 m/s).

Impact: Both players have $\sim 41\%$ more speed diagonally. The HJ solver computes value functions with these inflated capabilities, producing overly optimistic winning regions for *both* players. The net effect on game outcome depends on the specific configuration but is non-trivial.

Fix: Use `hj.sets.Ball` instead of `hj.sets.Box`:

```
control_space=hj.sets.Ball(center=jnp.zeros(2), radius=self.u_d_h)
```

Alternatively, project box controls onto the circular constraint in the online controller:

```
speed = np.sqrt(u_x**2 + u_y**2)
if speed > U_D_h:
    scale = U_D_h / speed
    u_x, u_y = u_x * scale, u_y * scale
```

4 Attacker Optimal Control

4.1 Attacker Reaching (T_{goal} Computation)

4.1.1 Paper (Section VI, Eq. 40–42)

The paper solves a separate reachability problem for the attacker:

$$\mathcal{R}_h = \{\mathbf{x}_A^h \mid \mathbf{x}_A^h \in \mathcal{T}\} \quad (13)$$

$$\mathcal{A}_h = \{\mathbf{x}_A^h \mid \mathbf{x}_A^h \in \Omega_{\text{obs}}\} \quad (14)$$

The earliest time to reach goal: $T_{\text{goal}}(\mathbf{x}) = \min\{t \mid \zeta_{\mathbf{x}}^{\mathbf{u}_A^*(\cdot)}(t) \in \mathcal{T}\}$.

T_{goal} is extracted from the value function by finding when $\Phi_{A,h}^{\text{reach}}$ changes sign during the backward solve (analogous to T_{capture} in Eq. 28).

4.1.2 Code Implementation

horizontal_solver.py:383-470: Solves attacker reaching with obstacles.

winning_conditions.py:25-59: Estimates T_{goal} :

```
def compute_T_goal(phi_a_reach, attacker_pos, time_horizon=10.0):
    value = interpolate_value(phi_a_reach, attacker_pos)
    if value <= 0:
        t_remaining = abs(value) / u_a_h
        return max(0.0, time_horizon - t_remaining)
    else:
        return float("inf")
```

4.1.3 Discrepancies

- **MODERATE: T_{goal} estimated via linear approximation.** The paper extracts T_{goal} from time-sliced value functions (finding the earliest sign change). The code approximates $T_{\text{goal}} \approx T - |V|/U_A^h$, which assumes a linear relationship between value magnitude and time. This is inaccurate for states far from the boundary or in the presence of obstacles.
Fix: Save time slices during attacker reaching computation (like `phi_z_time_slices.npz`) and use interpolation through slices to find exact T_{goal} .
- **MINOR: Attacker reaching solver missing wall constraints.** The solver includes static obstacles but does *not* include arena walls as avoid sets for the attacker, potentially underestimating T_{goal} for attacker positions near walls.

4.2 Attacker Escape Control (Online)

4.2.1 Paper

Horizontal (Eq. 21b): $(\mathbf{u}_A^h)^* = \arg \min_{\mathbf{u}_A^h} \left(\frac{\partial \Phi_h}{\partial \mathbf{x}^h} \right)^\top f_h$

Vertical (Eq. 26b): $(\mathbf{u}_A^z)^* = \arg \max_{\mathbf{u}_A^z} \left(\frac{\partial \Phi_z}{\partial \mathbf{x}^z} \right)^\top f_z$

4.2.2 Code Implementation

`attacker_node.py:263-329:`

```
# Horizontal: attacker minimizes (correct)
cmd.linear.x = -U_A_h if grad_xa >= 0 else U_A_h
cmd.linear.y = -U_A_h if grad_ya >= 0 else U_A_h

# Vertical: attacker minimizes (matches code convention)
cmd.linear.z = -U_A_z if grad_za >= 0 else U_A_z
```

4.2.3 Discrepancies

- **MINOR:** The paper says the attacker *maximizes* in the vertical game (Eq. 26b), while the code *minimizes*. This is consistent because the code's Φ_z has opposite sign convention from the paper's. Both result in the attacker moving *away* from the defender.
- **MODERATE: Box constraint for attacker.** Same as defender (Section 3.3) – the attacker's horizontal control space is box $[-U_A^h, U_A^h]^2$ instead of circular. This gives the attacker 41% more diagonal speed, overstating its capability beyond the already-conservative single integrator assumption.
- **MODERATE: Attacker uses game value functions for online control, not reaching value functions.** The attacker's online controller uses Φ_h (game value) and Φ_z to compute escape controls. But the paper's attacker (Eq. 42) uses the *reaching* value function $\Phi_{A,h}^{\text{reach}}$ for goal-directed control. The game value function makes the attacker play optimally against the defender, while the reaching function makes the attacker pursue the goal optimally. In practice, the game-optimal attacker is a harder opponent, so this is *conservative* from the defender's perspective.

5 Winning Condition Analysis

5.0.1 Paper (Section VI)

Proposition 1: If $T_{\text{goal}}(\mathbf{x}_A^h) \leq T_{\text{capture}}(\mathbf{x}^z)$, the attacker wins.

Theorem: Defender wins if $\mathbf{x}^z \in \mathcal{W}_{D,z}$, $T_{\text{goal}}(\mathbf{x}_A^h) > T_{\text{capture}}(\mathbf{x}^z)$, and $\mathbf{x}^h \in \mathcal{W}_{D,h}$.

5.0.2 Code Implementation

winning_conditions.py:144-234:

```
# Check: phi_h <= 0 AND phi_z <= 0 AND T_goal > T_capture
defender_wins = in_w_d_h and in_w_d_z and (t_goal > t_capture)
```

5.0.3 Discrepancies

- **CRITICAL: Winning condition compounds upstream errors.** Because Φ_h is computed with the wrong formulation (missing target region), $\mathcal{W}_{D,h}$ is inaccurate. The T_{goal} estimation is approximate. Together, these can cause incorrect game outcome predictions.
- **MODERATE: T_{capture} estimation.** `compute_T_capture` (line 62–90) uses the same linear approximation as T_{goal} . The more accurate `compute_T_capture_from_slices` exists (line 93–141) but is not used by default in `check_defender_wins`.

Fix: Use `compute_T_capture_from_slices` when time slices are available.

6 Wall Avoidance Safety Layer

Not in the paper. The code adds a wall-avoidance post-processing layer (`defender_node.py:137-191` and `numerical_sim.py:28-62`) that scales down or overrides optimal control commands near arena boundaries.

- **MODERATE:** This layer can **override the optimal game-theoretic control**, potentially breaking the theoretical capture guarantees. For example, if the optimal control commands the defender toward a wall to intercept the attacker, the safety layer may reduce this command, allowing the attacker to escape.

Mitigation: The paper’s formulation includes walls in the obstacle set Ω_{obs} , so the optimal control from Φ_h already accounts for wall avoidance. If the value function is correctly computed with wall constraints, the optimal control should never command the defender into a wall. The safety layer should only be needed as a numerical robustness measure (grid interpolation artifacts near boundaries), not as a primary wall avoidance mechanism.

7 Summary of Discrepancies

#	Severity	Description	Impact
1	CRIT	Horizontal game missing $\mathcal{T}$ in reach set	$\mathcal{W}_{D,h}$ doesn't account for attacker goal
2	CRIT	Box control constraints (not circular)	41% extra diagonal speed for both players
3	CRIT	$V_{h,T}$ in 4D without obstacles	$\mathcal{B}_h$ may include obstacle-collision states
4	MOD	$T_{\text{goal}}/T_{\text{capture}}$ linear estimation	Inaccurate timing, wrong game outcomes
5	MOD	$d_z^{\text{eff}}/d_h^{\text{eff}}$ threshold fallbacks	Weakened invariant set guarantees
6	MOD	Wall avoidance overrides optimal ctrl	Breaks capture guarantees near walls
7	MOD	Avoid set missing $\mathbf{x}_A^h \notin \mathcal{T}$	Capture inside target counts incorrectly
8	MOD	Attacker uses game VF not reaching VF	More adversarial than paper (conservative)
9	MINOR	PID fallback outside winning region	No theoretical guarantee (pragmatic)
10	MINOR	Dev grid resolution too coarse	Numerical inaccuracy (not a code bug)

Table 1: Summary of all identified discrepancies between paper and code.

8 Recommended Fixes (Prioritized)

8.1 Priority 1: Fix Horizontal Game Formulation

Files: `horizontal_solver.py`, `winning_conditions.py`

1. Add target region $\mathcal{T}$ SDF to the horizontal reach set (Eq. 5)
2. Add “attacker not in $\mathcal{T}$ ” condition to avoid set (Eq. 6)
3. Update winning region convention: $\mathcal{W}_{D,h} = \{\Phi_h > 0\}$
4. Re-derive optimal control signs for the corrected formulation

8.2 Priority 2: Fix Control Constraint Geometry

Files: `horizontal_game.py`, `vertical_game.py`, `horizontal_relative.py`, `attacker_reaching.py`

1. Replace `hj.sets.Box` with `hj.sets.Ball` for all horizontal control/disturbance spaces
2. Add post-hoc speed clamping in online controllers as a safety check
3. Vertical controls are 1D scalars – box constraint $|u_z| \leq U^z$ is correct as-is

8.3 Priority 3: Use 6D Horizontal Max Distance

Files: `horizontal_solver.py`, `compute_horizontal.py`

1. Make `solve_horizontal_max_distance_6d` the default pipeline
2. This is already implemented but not wired as default
3. Increases computation time substantially; consider for “medium” and “paper” presets only

8.4 Priority 4: Improve $T_{\text{goal}}/T_{\text{capture}}$ Accuracy

Files: `winning_conditions.py`, `horizontal_solver.py`

1. Save time slices during attacker reaching computation

2. Use `compute_T_capture_from_slices`-style interpolation for both T_{capture} and T_{goal}
3. Fall back to linear estimation only when slices unavailable

8.5 Priority 5: Address Wall Avoidance Integration

Files: `defender_node.py`, `numerical_sim.py`

1. Ensure walls are properly included in Ω_{obs} for the HJ solver
2. Reduce aggressiveness of safety layer (use as last-resort, not primary avoidance)
3. Consider removing safety layer entirely once value functions are correctly computed with walls

9 Conclusion

The codebase implements the core algorithmic framework from the paper (HJ decomposition, reach-track control, invariant capture sets) with generally correct dynamics and solver integration. However, three critical discrepancies – the missing target region in the horizontal game, box vs. circular control constraints, and the 4D tracking value function without obstacles – likely explain the observed failing behaviors.

The most impactful fix is correcting the horizontal game formulation (Priority 1), which would produce a value function that properly accounts for the interaction between capture and goal-reaching. Combined with fixing the control constraints (Priority 2), these changes should significantly improve game outcomes and match the paper’s theoretical guarantees.